Microprocessors and Microsystems 94 (2022) 104671

Contents lists available at ScienceDirect

Microprocessors and Microsystems

journal homepage: www.elsevier.com/locate/micpro

Early RTL delay prediction using neural networks

Daniela Sánchez Lopera *, Lorenzo Servadei, Sebastian Prebeck, Wolfgang Ecker

Infineon Technologies AG, Germany
Technical University of Munich, Germany

ARTICLE INFO

Keywords:
Register Transfer Level
Machine Learning
Delay
Slew
Static Timing Analysis

ABSTRACT

Nowadays, the digital chip design flow starts with formal specifications, which are mapped to Register Transfer Level (RTL) models using different underlying (micro-) architectures. By doing so, a hardware designer predicts and resolves time-critical parts to achieve an RTL-design that meets all constraints after synthesis. However, wrong predictions can be detected only later in the design flow, thus leading to long design iterations. Classical methods estimating delay in early design stages are constrained to the type of components or are computationally expensive for larger designs. This paper proposes a machine learning-based approach to estimate pin-to-pin delays for RTL combinational circuits. To improve the quality of the predictions we combine slew and delay estimation. To that end, training data are built using features of components generated by a model-driven hardware generator framework. Ground truth labels for delays, slews, and their interdependencies are extracted using open-source tools for logic synthesis and static timing analysis. Two different datasets are built: one targeting logic gates and multiplexers and an enlarged one, which generalizes to more RTL primitives A model trained using the former dataset achieves, on average, a coefficient of determination R^2 of 87% when evaluating over 4-bit prefix adders. Using the enlarged dataset, the best model reaches an R^2 of 77%. On average, our models are 8.4× faster w.r.t the time required to run synthesis and timing analysis. Results show that generalizing to more primitives decreases the models' performance, but the runtime benefit is maintained. Based on the delay estimation, critical areas of the design can be detected and proper microarchitecture decisions can be taken earlier and faster in the design flow.

1. Introduction

The chip design flow is a complex and iterative process. First, informal specifications are mapped to system-level descriptions. Those are further translated into functional, logical, and physical designs using Electronic Design Automation (EDA) tools. Hereby, synthesis tools optimize the gate-level netlist to meet timing constraints while minimizing area and power consumption [1]. Before fabrication and packaging, the correct functionality of the design has to be verified, and the compliance of design metrics in terms of Power, Performance, and Area (PPA) has to be validated. Principally, designs must meet timing constraints and work at the desired clock frequency. Although transformation and optimization have been automated using EDA tools, it is the task of the hardware designer to map initial specifications to Register Transfer Level (RTL) models that intentionally meet the timing requirements. This mapping is challenging, as RTL models can be built using multiple different implementations or (micro-) architectures, whose timing behaviors are known only after costly synthesis and timing analysis steps.

Early estimations of the delay information of a design at RTL can ease the design-space exploration to find the optimal implementation and architectural option. Smarter decisions in earlier design stages result in fewer iterations of the complete EDA flow, higher Quality of Results (QoRs), and potentially, less time-to-market. In particular, Hardware (HW) generator frameworks can benefit from earlier predictions, which could be used to evaluate implementation decisions during the generation without any human-in-the-loop.

Classical methods estimating delay from a higher level of abstraction use polynomial, macromodeling, and analytical approaches. They have several drawbacks: they are slow, constrained by the type of components, computationally expensive, or give only a very rough estimation. In the last decade, ML has proven its ability to support classification and prediction tasks in many fields with higher accuracy than classical methods. Consequently, it is being incorporated into the EDA flow, especially in logic synthesis, placement, and verification [2].

This paper proposes Neural Networks (NNs) for pin-to-pin component delay estimation of RTL components in Application-Specific

* Corresponding author at: Technical University of Munich, Germany.
E-mail addresses: daniela.sanchezlopera@infineon.com, daniela.sanchez-lopera@tum.de (D. Sánchez L.).

https://doi.org/10.1016/j.micpro.2022.104671
Received 27 April 2022; Received in revised form 27 June 2022; Accepted 29 August 2022
Available online 9 September 2022
0141-9331/© 2022 Elsevier B.V. All rights reserved.

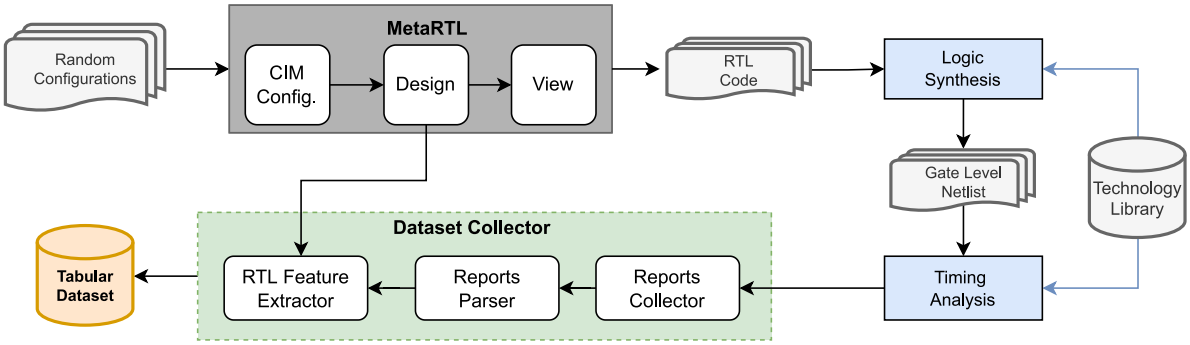


Fig. 1. General flow for dataset collection.

Integrated Circuit (ASIC) designs. Extending the discussion in [3], the main contributions of this paper are:

(1) We propose a solution for delay estimation using NNs from RTL targeting ASIC technology.
(2) The proposed approach is reproducible for any RTL design using different types of primitive components. In contrast to [3], an enlarged training dataset is built to cover logic gates and multiplexers, but also demultiplexers, branch, and arithmetic operators. The algorithm uses features from RTL and labels from EDA tools.
(3) To improve performance, our models learn delay and slew information simultaneously.

In this paper, the training data are built using a model-driven HW generator framework. Targeting a specific CMOS ASIC technology, component features and ground truth labels are extracted using open-source EDA tools. Fig. 1 shows the complete data collection flow. ML models using one-hot encoded and embedding feature vectors are trained and fine-tuned. The trained models are used to get estimated pin-to-pin delay values. To improve the performance of our models, slew information is included in the feature set. Moreover, we sequentially predict slew and delay values per component and then propagate the predicted slew along the delay path.

To evaluate the models' performance, hardware adders are used as proof of concept. Specifically, three different architectures of 4-bit prefix adders with three different input transition times are compared. The trained models were used to predict all the pin-to-pin delays of all primitive components in the designs. When training a model with a dataset of logic gates and multiplexers, a simple Multilayer Perceptron (MLP) using one-hot encoding achieves, on average, a coefficient of determination R^2 of 87% w.r.t delay values from the EDA tool. The MLP model using embedding feature vectors achieves an R^2 score of 79%. Training with an enlarged dataset, the best results are obtained by the MLP architecture using an embedding input layer. This approach reaches, on average, an R^2 of 77% and a Mean Squared Error (MSE) of 0.68%. Both models are around 8.4× faster than the EDA tools.

The results show that generalizing to more component types decreases the R^2 score. Nevertheless, RTL designs are not solely composed of logic gates and multiplexers. Thus, using the enlarged dataset is considered the most viable approach. Compared to EDA tools, our approach represents a good trade-off between the quality of the predictions and runtime.

This paper is organized as follows: Section 2 briefly reviews concepts related to Timing Analysis (TA) and to ML applied to regression tasks. Section 3 provides an overview of the delay estimation state of the art. Section 4 describes our ML solution. Section 5 provides details of the benchmark circuits used to evaluate our trained models. The experiments and results comparing models trained on both datasets are summarized and discussed in Section 6. Finally, Section 7 concludes by mentioning open challenges, application fields, and future directions.

2. Background

This section reviews background concepts related to TA in the digital design flow, and building blocks of an ML solution applied to a regression task.

2.1. Timing Analysis (TA)

TA is an EDA tool to determine whether a design works properly at the required clock period. TA can be dynamic or static. The dynamic analysis simulates the complete circuit using input vectors and evaluating the outputs. STA is a static approach, as it propagates the gate delays from primary inputs to outputs. The gate delay is the time needed for a signal to propagate through a gate, i.e., going from input to output port [4].

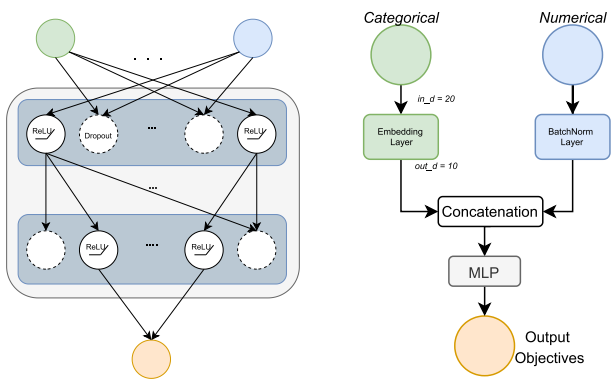
In STA, all paths in a design are analyzed using gate delays and signal arrival times, especially the path with maximum and minimum delay. STA provides a severity measure called *slack*: a negative slack means that the signal through the path is too late, a positive slack means that the cells in the path could be replaced by ones with higher delay (and potentially, lower area) without causing timing violations [5].

STA highly depends on the delay values used for each gate in the whole design. STA tools obtain the delay values using a Non-linear Delay Model (NLDM). Specifically, the delay of a cell is defined as a Lookup Table (LUT) with two variables as indexes: input *slew* and *fanout* load. The former is the time that a signal requires to make a transition. If the signal changes from low to high, it is called a *rising* edge. From high to low, it is called a *falling* edge. The fanout is the capacitive load at the output [4]. For a constant output load, an increase in the input slew will reduce the delay. For a constant input transition, the delay will increase when the fanout capacitance increases. The slew is propagated along the circuit. The output slew of a cell is the input slew of the next connected cell in the timing path. Output slews are also defined via LUTs with input slew and fanout load as indexes. An NLDM stores delay and slew LUTs in a technology library as the *Synopsis Liberty* file [6]. STA tools can read those liberty files. During TA, the tool uses interpolation or extrapolation to calculate delays and output slews for index values that are not specified or outside the defined index tables. Moreover, STA tools are also able to scale the delay and slew values according to the current operating conditions, which physically affect the cell behavior such as temperature, voltage, and process [1].

In this paper, an NLDM is employed for extracting ground truth labels for the delay and slew values. The prediction of these metrics is defined as a multi-objective regression problem that can be solved using ML.

2.2. ML for regression tasks

In supervised learning, models are trained to learn the relationship between data features and ground-truth labels for regression or classification. In this paper, a model is trained to solve a regression task. The building blocks required to solve such a task using ML are:



(a) MLP (b) TabMLP

Fig. 2. Implemented ML Models.

2.2.1. Data collection

In a regression problem, the feature vectors v_i per row i are labeled with the n target objectives y_n. The goal is to train a model that, given an unseen feature vector v, predicts the quantitative values y.

2.2.2. Feature analysis

To construct meaningful datasets, statistical approaches for feature analysis should be used to analyze the correlation and importance of selected features:

2.2.2.1 Spearman's Rank Correlation Coefficient (SRCC): One method to analyze discrete and continuous variables is the correlation between features. A special case of correlation is called the *SRCC* [7]. It measures how well the relationship between two variables can be represented by a monotonic function, which may be linear or not. An SRCC of one indicates that two variables increase in value together. An SRCC of -1 corresponds to a decreasing monotonic trend: as one variable increases, the other variable decreases.

2.2.2.2. Mutual Information: The MI [8] between two variables X and Y measures any dependency between them. A MI of zero indicates that the variables are independent. In other cases, the MI is a positive value, where higher values indicate higher dependency. To quantify the impact of X on the fluctuation of Y, the MI is represented as the sum of the conditional entropy of X given Y and Y given X. The entropies are estimated using K-Nearest Neighbors (KNNs) [9] distances over the permutation of the features X. KNN is a non-parametric method that searches through the training set the k-closest instances to an unseen data point and returns the closest data as the prediction.

2.2.2.3. F-Test: Contrary to MI, F-Test [10] captures only the linear dependency between two random variables, i.e., estimates their degree of linear dependency. In linear regression tasks, F-Test is a number in the range of [0, 1] and measures how relevant independent variables are. To that end, the correlation between each feature in X and Y is calculated. Then, the F-statistic is applied over the number of samples.

2.2.2.4. Shapley values: Shapley values [11] are a concept from game theory that is being used for explainable and interpretable Artificial Intelligence (AI) [12]. Shapley regression values measure feature importance for linear models, i.e., compute which features contribute more to a prediction. To that end, two different models are trained with and without a chosen feature, respectively. The performance of both models is compared, and the differences are computed for all possible feature subsets. The final Shapley values are the weighted average over all possible subsets [12].

2.2.3. Model selection

Once the training data and their features are selected, the architecture of the ML model is defined so that the last layer outputs the predicted vector. In this paper, an MLP [13] is used due to its simplicity and good performance in multi-objective tasks. A simple MLP can represent a wide variety of functions when using appropriate parameters. Fig. 2(a) sketches the main layers of an MLP: input, hidden, and output layer. The input layer is made of as many neurons as the number of input features. For all our models, the number of hidden layers, as well as the number of nodes for those, are hyperparameters. The output layer has as many neurons as the number of objectives. The modules are fully connected, i.e., the value of each element of the subsequent layer is the value of the so-called *activation function* applied to the weighted sum of all the values of the current layer. The NN uses the backpropagation learning method to update all weights during training.

The input features for the first layer should be numerical or one-hot encoded. Another alternative is to use embedding vectors [14]. Embedding vectors are widely used in Natural Language Processing (NLP) to embed words and to learn similarities between them [15,16]. Feature embeddings alleviate two main issues of one-hot-encoded vectors: high dimensionality and sparsity. Embedding layers are trained to learn the optimal mapping of a unique entry to a vector of a desired dimension.

2.2.4. Hyperparameter search

Hyperparameters are model parameters that cannot be trained. Using optimal hyperparameters improves the model performance. Thus, hyperparameter search should be conducted. Manual tuning, grid search, and random search are considered brute-force methods. Grid and random search perform better than manual tuning, but they do not consider past evaluations. Hence, they lose significant time by evaluating hyperparameters in the same inadequate region [17]. Therefore, a model-based sequential method called Tree-Structured Parzen Estimator (TPE) was introduced [17]. TPE creates a surrogate model of the objective function and considers past evaluations to choose new hyperparameters. This work uses TPE to fine-tune all models.

2.2.5. Model evaluation

The models using optimal hyperparameters are trained. During training, validation and testing, the performance of the model should be assessed. In a regression task, this performance can be measured using the Mean Absolute Error (MAE), MSE, and the coefficient of determination R^2. The MAE in Eq. (1) is the average of the absolute difference between real and predicted values, where y_i is the ground truth and $\hat{y}_i$ the predicted. MSE averages the squared value of that difference as shown in Eq. (2). MAE and MSE are suitable for datasets with outliers. The lower the values of MSE and MAE, the higher the prediction accuracy has the regression model. The R^2 in Eq. (3) where $\bar{y}$ is the mean value, measures how well the data fits the model. The higher the R^2 value, the better the model is. The R^2 can also be negative, indicating that the mean is a better estimator than the prediction.

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|, \tag{1}$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2, \tag{2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y}_i)^2}, \tag{3}$$

3. Related work

The NLDM is used by STA tools when the gate-level netlist is available. From a higher level of abstraction, other methods estimate the delay without using EDA tools:

3.0.1. Analytical

Analytical methods attempt to relate delay to other features of the design.

The *Logical Effort (LE)* method [18] relates the delay of a CMOS circuit to the LE value, which is defined as the ratio of the input capacitance of a gate, and the input capacitance of an inverter delivering the same output current. It is estimated as counting transistor widths for the gate divided by transistor widths for the inverter. The gate delay is defined as the sum of the effort delay and parasitic or zero-load delay. The effort delay is the LE multiplied by the fanout. The LE method allows comparing different implementations without running synthesis, it is technology-independent, and it can be used in multi-stage designs. However, it does not consider the effect of transition times of the inputs, and it uses estimations for the parasitic delay and LE values.

In [19], four analytical novel methods are introduced to estimate delay given the placement of a circuit. Using the same technology and same router, base model designs are used to extract factors such as net degree, route length, net delay, and route Steiner tree length. Those factors are terms of the estimation functions, which can be quadratic or linear equations. For instance, the proposed *Delay Sampling* method computes delay as a quadratic function of the pin-to-pin placement length. With base designs, the coefficients are computed, and for new designs, the equation is used with the current wire's length. This method uses scaling factors and LUTs to link the estimated variables per net. Thus, it can be slow and computationally expensive for larger designs.

In [20], the delay is estimated as a logarithmic function of the complexity of the gate and the fanout. Type of gate and fanout values are extracted from the designs before logic synthesis. These are scaled by constant coefficients or technology-dependent factors calculated using the NOT and NAND delay, the fanout limit, and the standard load. In this paper, the authors did not consider the effects of the signal transition times on the estimation.

In [21], the delay is approximated as a linear function of the number of programmable interconnect points and the number of segments between two circuit points. Scaling coefficients were computed from observations done using the Xilinx timing analysis tool. The approximation function estimates delay for Field Programmable Gate Arrays (FPGAs) considering also the wiring effects, but it does not support ASIC.

3.0.2. Macromodeling

Macromodeling is a usual way to estimate design metrics from a high-level circuit description. It can be seen as a template, in which the quantity under analysis is related to a set of parameters and coefficients. The set of parameters is computed by using a set of base designs. Commonly, linear regression is used to define the actual equation of the model.

In [22], a macromodeling technique is used to estimate the delay at the behavioral level. Technology-independent gate-level netlists are mapped to Binary Decision Diagrams (BDDs). For delay estimation, the model relates various types of the complexity of a Boolean function to its gate-level implementation as size, number of inputs, and density.

In [23], a macromodel for delay estimation at RTL is proposed as a function of V_{dd} and V_t. The two scaling coefficients are extracted from experiments in HSPICE. However, the model only supports NAND, NOR, and NOT gates.

3.0.3. Polynomial-based

Polynomial models are regression-based, where the delay is the dependent variable, and design parameters are the independent variables.

QUEST [24] is a tool that uses *best-fit* polynomial estimation for area and delay from RTL descriptions. QUEST transforms the RTL code to a technology-independent form, similar to what a logic synthesis tool does. But it only targets three logic gates: AND, OR, and NOT. The initial parameter extraction for the best-fit polynomial uses few test designs.

Table 1
Selected features and objectives.

Feature/Objective	Description
Component type (X_0)	One of T[a] possible types
Input number (X_1)	Current input pin considered
Number of inputs (X_2)	Random number $\in [1, B]$[b]
Bit width inputs (X_3)	Random number $\in [1, 32]$
Bit width output (X_4)	Random number $\in [1, 32]$
Connections to input (X_5)	Connected components $\in [0, 6]$
Connections to output (X_6)	Connected components $\in [0, 6]$
Slew In $[fall, rise]$ (X_7)	Transition times for current pin
Number of outputs[c] (X_8)	Random number $\in [1, 32]$
Sum bit width inputs[c] (X_9)	Sum over all inputs bit width
Avg. bit width inputs[c] (X_{10})	Average over all inputs bit width
Slew Out $[fall, rise]$ (Y_1)	Transition times for output pin
Delay $[fall, rise]$ (Y_2)	Delay from current to output pin

[a]If training set is DS-I, then $T = 20$. Otherwise, $T = 37$.
[b]If training set is DS-I, then $B = 12$. Otherwise, $B = 32$.
[c]Extra features considered for DS-II.

In [25], a one-shot scheme predicts the area-delay curve for Boolean functions. This method proposes a novel *delay complexity* measure based on the *on and off-sets* of a Boolean function. According to the authors, the size and the number of prime implicants of a function affect the time needed for evaluating the function. An STA tool is used to generate a family of curves of delay values versus the novel complexity measure. This method provides a minimum and maximum delay estimation. However, it does not support circuits with XOR and other complex gates.

3.0.4. Machine learning-based

In [26], an ML-based delay estimator is used during FPGA placement. They use ML for three main reasons. First, training, validation, and inference are faster than running traditional EDA tools. Delays are non-linear functions in which ML has proved high performance. Finally, an ML-based estimator can also be easily updated. They build a dataset with 20 features, where the most relevant are: number of pins, Half Perimeter Wire Length (HPWL), and average congestion.

Our paper is comparable to [26], as we defined the delay estimation as a regression problem. However, we estimate the delay at RTL and use delay training data after logic synthesis rather than after placement to avoid learning inefficient mappings done during physical synthesis. [26] targets FPGA, not ASIC technology, therefore they did not consider any effect of slew. [26] uses Random Forest (RF), and we use NNs, a more expensive method that can generalize better than RF when using appropriate parameters. To further improve, we perform a hyperparameter search. Compared to other methods, our approach can be used for all types of logic gates and more complex components. Moreover, the modeling part is easier, as only the dataset has to be generated rather than modeling the delay gate behavior with statistical or mathematical functions.

4. Method

Gate delay estimation is challenging due to the many dependencies: technology, operating conditions, cell types, outputs capacitance, and signal transition times. Therefore, we proposed a new approach to estimate cell delays using NNs. We define delay pin-to-pin estimation as a multi-objective regression problem. To get a more accurate delay estimation, the slew per gate is also predicted and propagated. The input for the regression model is a feature vector $\mathbf{v_i}$ where i is a pin-to-pin connection of a component within a design. The vector $\mathbf{v}$ encodes information of the features from Table 1. The output of the model $\mathbf{y_i}$ is a vector of four components representing the regression objectives: $[slew_out_fall,\ slew_out_rise,\ delay_fall,\ delay_rise]$

Table 2
Component types covered in dataset DS-II.

Category	Type	#Circuits	#Samples
Logical[a,b]	AND, NAND, OR, NOR, XOR, XNOR NOT	200.9k	2564k
Arithmetic	CMULT, CDIV CMOD, CREM, CPLUS, CMINUS, HWPLUS, HWMINUS HWMUL	68.6k	665k
Multiplexing	MUX[b] , DEMUX	1.0k	78k
Branch	GT, GTEQ, LT, LTEQ, EQ, NEQ, ISPOS	5.5k	126k

[a]Considering three fashions: logical, reduced and bitwise.
[b]Component type also considered in DS-I.

Table 3
STA configuration parameters.

Setup	Name	Value	Unit
Ports	Driv. Cell Load	0.0015	fF
	Input Delay	2.1	ns
	Output Delay	0.7	ns
	Input Load	0.0075	fF
	Output Load	0.015	fF
	Max Input Slew	Varies[a]	$[rise, fall]$
	Min Input Slew	Varies[a]	$[rise, fall]$
Clock	Period	6.6	ns
	Ideal Period	7.00	ns
	Latency	[0.1, 0.2]	$[min, max]$
	Transition	[0.1, 0.2]	$[rise, fall]$
	Uncertainty	[0.6, 0.15]	$[setup, hold]$
Operating conditions	Voltage	1.08	V
	Temperature	−40	°C
	Process	1.30	–
Analysis	Corner Case	Worst	–
	Delay Model	LUT	-

[a]Options: $[0,0]$, $[0.5,0.5]$, $[1,1]$, $[1,1]$.

4.1. Data collection

Fig. 1 sketches the whole flow for dataset generation. Our proposed method is independent of any RTL implementation. However, we use a HW generator framework, which generates our designs in a faster and automated way by using meta-modeling. The used HW generation framework is *MetaRTL* [27]. It applies the Model Driven Architecture (MDA) principle to RTL generation [28]. The abstract model (Computation Independent Model (CIM) in MDA) hierarchically describes configuration options.

4.2. Feature selection

In this paper, different features are selected from two different datasets. We generate random configurations of components to build two different training sets: A simple dataset (DS-I) and a generalized dataset (DS-II).

4.2.1. DS-I

In this dataset, twenty types of RTL components are used to build the training designs: multiplexers, inverters, and logic gates (AND, NAND, OR, NOR, XOR, and XNOR). The latter in a bitwise, logical, and reduced fashion. In DS-I, each design contains only the above-mentioned RTL components, with a randomly generated number of inputs and bit widths. The number of inputs varies from one up to 12. Input bit widths range from one to 32. To consider different fan outs, the number of components per design ranges from zero up to six. Single-component designs have zero connections to the output pin. Additionally, two different transition times of primary input ports are considered during the TA. Our collected dataset DS-I consists of $42.9K$ samples.

4.2.2. DS-II

DS-I is extended to cover other component types such as demultiplexing, arithmetic, and branch operators. This enlarged dataset is what we name DS-II. Table 2 lists all 37 types of primitive components covered. Additionally, input bit widths range from one to 32 for one up to 32 inputs.

Different component types have different ranges for the selected RTL features. For instance, a branch operator component such as GT has two inputs and one output per definition, while for logic gates, the number of inputs is at least one. Thus, in the generated random configurations, the number of inputs for *GT* is fixed to two, while for logic gates is in the range of $[1, 32]$. This results in an imbalanced dataset w.r.t. the component type as shown in Table 2, where the number of circuits using logical gates is 36.5× larger than the number of circuits for branch operators. In total, our collected dataset DS-II consists of $3.4M$ samples.

For both datasets, the generated Verilog code is synthesized using Yosys,[1] [29] an open-source tool that reads Verilog Hardware Description Language (HDL) files and creates a gate-level netlist by mapping it to CMOS cells of the technology library. We use a 40 nm technology for our experiments. The synthesized netlists are the required input to the STA. In this work, OpenSTA[2] [30] is used, an open-source version of the commercial Parallax timer tool. A technology library configuration is chosen, and the tool is set up with parameters exemplarily shown in Table 3.

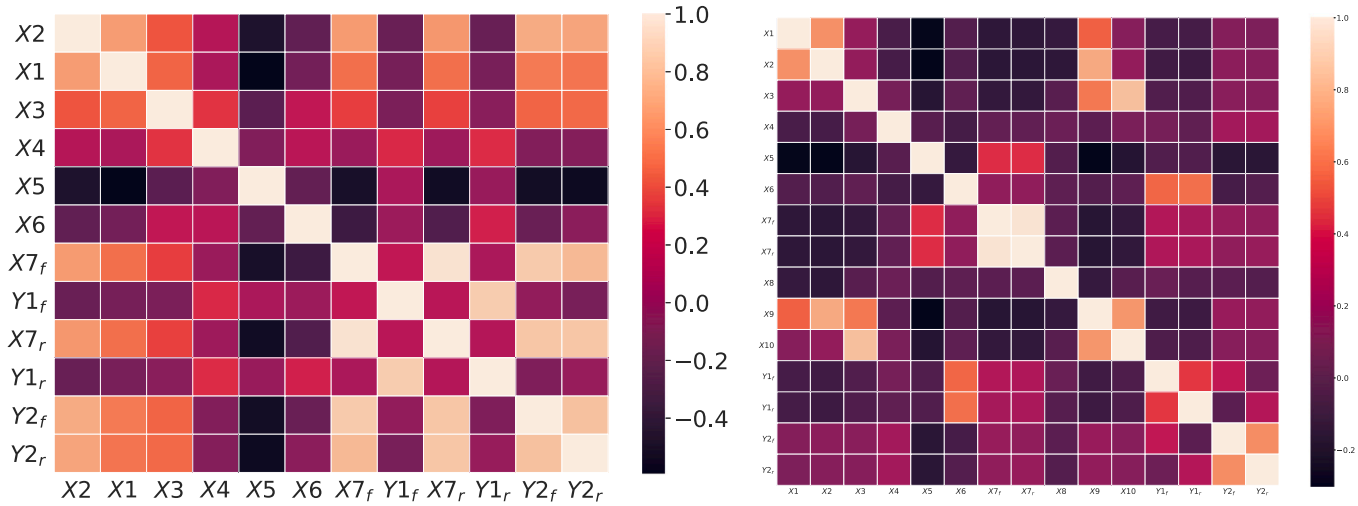
To consider variations of the slew, we generate timing reports with two and four different variants for the transition time of the input ports for the datasets DS-I and DS-II, respectively. The timing report style is configured using TCL scripts. Pin-to-pin timing reports are collected for all components within a design. To that end, the tool is configured to analyze all the four possible timing paths i.e. *Register-to-Register*, *Register-to-Output*, *Register-to-Input* and *Input-to-Output*, and the maximum edges: *max rise* and *max fall*. Automatically, we read the generated reports and extract information related to arc type, slew, and delay. These are used to find the maximum rise and fall delay and the corresponding slew values.

RTL features and information from EDA tools are mapped to a tabular dataset. Each row of the dataset or sample represents a pin-to-pin connection of an RTL component inside a circuit. Table 2 shows the number of samples per component category. The columns of the dataset contain the RTL features as well as ground truth labels. The generation of our dataset following the steps shown in Fig. 1 took around three weeks when running in a CPU machine.

Table 1 lists our feature set X and objectives Y. In this subsection, the motivation of our selection is presented:

(1) X_0: It gives an intuition about the gate's complexity. It also correlates to implementation patterns (e.g., the transistor netlist).
(2) X_1: To distinguish the pin-to-pin connection from the current pin to the output. It reflects the signal's path through the design, especially if inputs have a different function (e.g., multiplexer).

[1] https://github.com/YosysHQ/yosys.
[2] https://github.com/The-OpenROAD-Project/OpenSTA.



(a) DS-I (b) DS-II

Fig. 3. Spearman's Correlation (SRCC) Matrix.

(3) X_2, X_3, and X_4: The time needed to execute a function depends on the number of inputs and their sizes. The number of inputs correlates to the implementation pattern.
(4) X_5 and X_6: Gate delay depends on the fanout, which is proportional to the number of connections.
(5) X_7: Gate's output slew is dependent on the slew model, which is also a function of the input slew and the fanout. The slew for primary ports is configured in the STA tool. The slew for other connections is propagated.
(6) X_8: The number of outputs correlates with the component implementation. It should be considered for multiple output components (e.g., demultiplexers).
(7) X_9, X_{10}: The bit width of other input pins different from the one indicated with X_1 gives information about the complexity of the logic mapping.

In both datasets, the data types of the selected features are mostly integers or floating-point values. For the component type X_0, the data type is a string with the name of one of the component types supported.

4.3. Feature analysis

This paper uses the SRCC over both datasets. Moreover, we calculate the MI, F-test, and Shapley Values for DS-II, specifically for the features X and the two objectives: output slew (Y_1) and delay (Y_2) distinguished into rising (r) and falling edge (f).

For simplicity, only the results for the falling edge, i.e., Y_{1f} and Y_{2f} are shown. The results for the rising edge highlight similar trends described below. The only difference is that the dependence between the input slew feature for the rising edge X_{7r} and objectives with same edge type, i.e., Y_{1r} and Y_{2r}, is stronger than with the falling edge X_{7f}. In the following feature analysis, the categorical feature X_0 was not included.

4.3.1. SRCC:

Fig. 3 shows the SRCC matrix pairwise calculated for both datasets. For DS-I, all the features X_i correlate with the objectives Y_2. As expected, there is a high correlation between Y_2 and X_7 features and with structure-related features such as X_1, X_2, and X_3. Similarly, the features X_i correlate with the objectives Y_2 for DS-II. As expected, there is a high correlation between Y_1 and X_6 features. The lower correlation between X_5 with other features and objectives is because only one component type of the dataset has multiple outputs. For the remaining component types, the number of outputs is fixed to one.

4.3.2. MI and F-test:

Fig. 4 shows the MI (gray bars) and F-test statistic (orange bars) between the selected features. Fig. 4(a) shows the highest degree of linear dependency measured by the F-test is between the output slew and the features input slew X_7 and fanout X_6, while the MI is the highest for features related to input bit widths X_9 and X_{10}. Similarly, the MI between delay and X_9 and X_{10} are the most significant, as shown in Fig. 4(b). Moreover, the F-test indicates that the most relevant linear dependency occurs between the delay and the output bit width X_4.

4.3.3. Shapley values:

The Shapley values highlight the impact of each selected feature in the prediction of each objective. To that end, an XGBoost [31] regressor model is used as an explainer. Fig. 5(a) shows that the most significant features for the output slew prediction were related to the fanout X_6 and the input slew X_{7f} as expected. For the delay prediction, RTL features such as the bit width of the output port X_4 and the number of inputs X_2 have higher Shapley values, as shown in Fig. 5(b). The input slew is again among the top five most important features.

4.4. Neural networks

4.4.1. Pre-processing

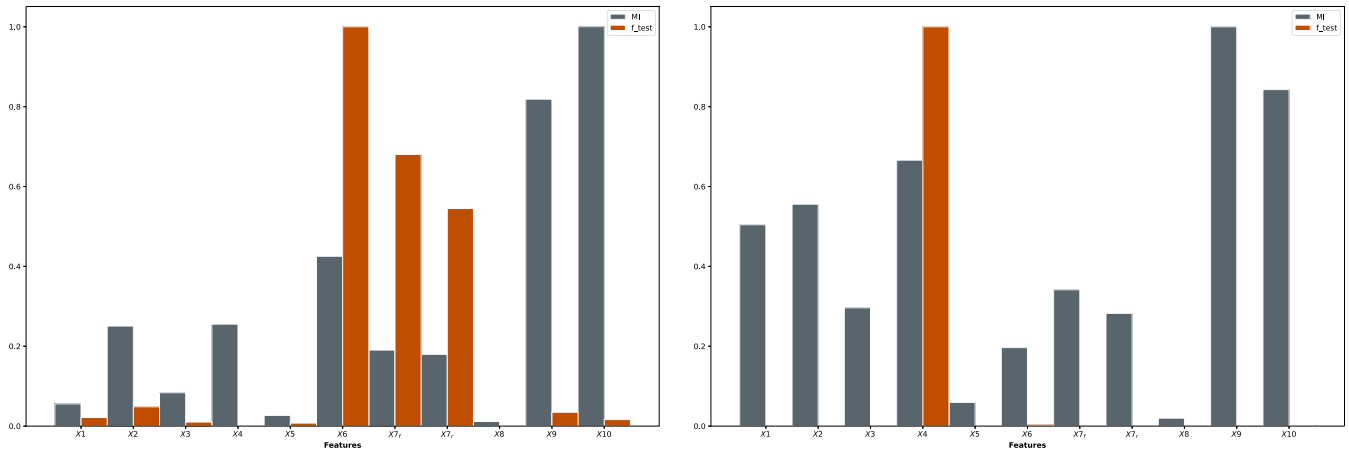
In both datasets DS-I and DS-II, each sample is a pin-to-pin connection of a randomly generated design using different variants for the input transition times.

For both datasets, the categorical feature X_0 is one-hot encoded. X_0 is mapped to an $N \times T$ sparse matrix, where N is the number of samples and T the number of component types supported, i.e., 20 for DS-I, and 37 for DS-II. Additionally, we map the bit widths per input to a zero-padded vector, resulting in an $N \times B$ matrix, where $B = 12$ and $B = 32$ for DS-I and DS-II, respectively. A *MinMaxScaler* is used to scale the four objectives in the range $[0, 1]$.

Both datasets were split into training (60%), validation (30%), and testing (10%) sets.

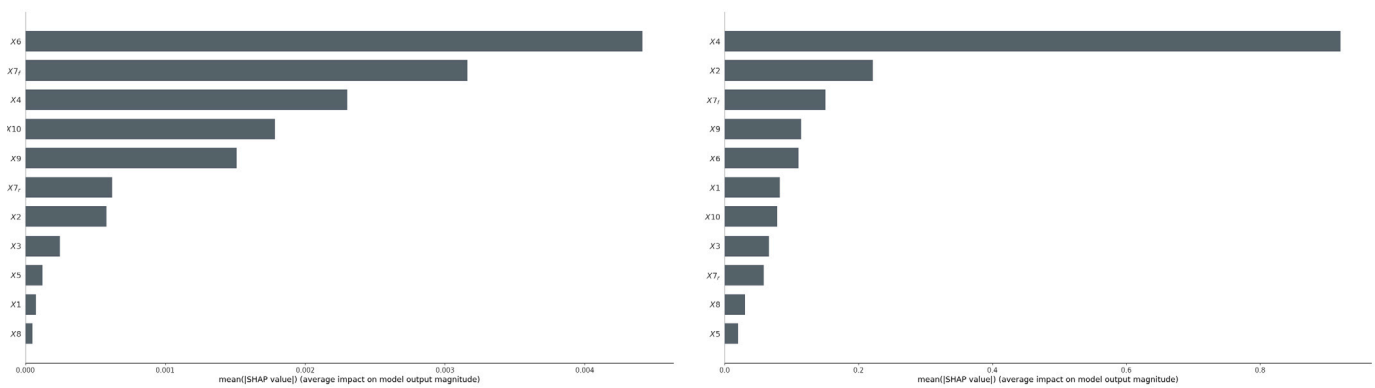
4.4.2. Architectures

This paper compares two main architectures: a simple *MLP* and the *TabMLP* architecture from [32]. The simple MLP architecture is sketched in Fig. 2(a). Each hidden layer uses a ReLU activation function with a dropout layer to avoid overfitting. Training the MLP model with DS-I, the number of input neurons is 41 and 43 with and without input slew features, respectively. Input slew features are always considered



(a) Slew Out Y_{1f} (b) Delay Y_{2f}

Fig. 4. MI (bars colored in gray) and F-Test (bars colored in orange) values for falling edge in DS-II. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)



(a) Slew Out Y_{1f} (b) Delay Y_{2f}

Fig. 5. Shapley values for falling edge in DS-II.

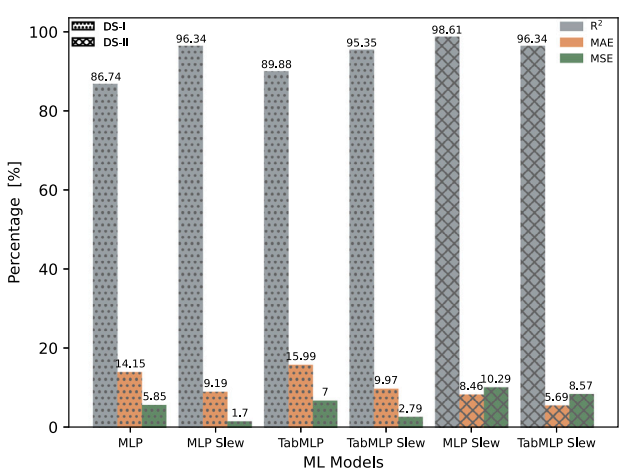


Fig. 6. Avg. results of estimation in test set for DS-I and DS-II.

when training with DS-II. As the supported component types and the maximum number of inputs increased, the input layer of the MLP model has 80 neurons.

The TabMLP architecture in Fig. 2(b) uses an embedding layer encoding the categorical feature X_0 into a D-dimensional vector, which is concatenated with the other features and passed through an MLP model. For DS-I, D is equal to 10. The total number of numerical input features is 21 and 23, with and without considering the slew, respectively. The number of neurons in the output layer is two when predicting rise and fall delay or four when including the prediction of output slew for the same edges. For DS-II, D is 19. Due to the new supported ranges and the consideration of slew features, the total number of numerical input features is 43, and the output layer has four neurons.

By using the two mentioned ML model architectures, we build six models: For DS-I, we train four models: an MLP, and TabMLP, with and without considering slew. For DS-II, we employ the same architectures but always consider slews. Using the TPE implementation from [33], the best set of hyperparameters for each model is found and shown in Table 4.

4.4.3. Training, validating and testing

The models were trained in a Nvidia Tesla P40 GPU for 300 epochs. For the MLP model, the overall training time when using DS-I was 164.9 s, and for the TabMLP model 706.5 s. For DS-II, the training times were 3.7 h and 4.4 h for the MLP and TabMLP models, respectively. Fig. 6 shows the results when evaluating each model over the test dataset. The test dataset consists of generated designs using one gate with zero, up to six connected gates. As shown, the models achieve an accuracy higher than 86% even without considering slew features nor propagation.

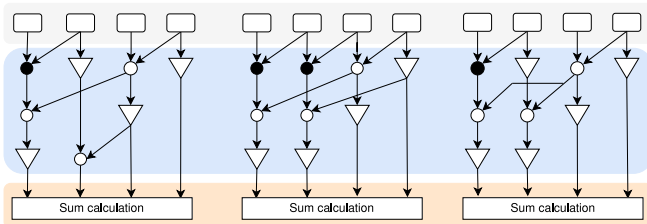


Fig. 7. From left to right: BKA, KSA, SKA. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 4
Best hyperparameters.

Dataset	Models	Slew	No. layers	No. neurons per layer	Dropout per layer	Learning rate
DS-I	MLP	✗	3	[39, 17, 62]	[0.04, 0.01]	0.008
		✓	2	[111, 59]	[0.07, 0.08]	0.040
	TabMLP	✗	3	[210, 145, 84]	[0.12, 0.11]	0.027
		✓	3	[144, 102, 74]	[0.17, 0.24]	0.029
DS-II	MLP	✓	2	[256, 255]	[0.037, 0.27]	0.001
	TabMLP	✓	3	[256, 225, 128]	[0.22, 0.32]	0.005

Table 5
Implementation details for the three adders.

Circuit	Number				
	BXORs	BANDs	BORs	Pin-to-Pin connections	Max. distance
BKA	8	7	3	36	7
KSA	8	8	4	40	9
SKA	8	13	8	58	9

5. Application

Adders are a key component in ASIC designs. The simplest version of an adder is the Ripple Carry Adder (RCA). An N-bit RCA is built using N full adders, the carry-out bit of the previous stage becomes the carry-in of the current stage. At any particular time, only one full adder instance is used [34]. More complex architectural versions are the *Parallel Prefix* adders, as they split the computation into three stages: Pre-processing, Carry-propagation, and Post-processing [35]. Three main architectures of parallel prefix adders are shown in Fig. 7: SKA, BKA, and KSA. In the gray-colored stage, rectangles calculate the *propagate* and *generate* bits for primary inputs. In the blue stage, generate and propagate values per bit are calculated using: generate blocks (circles), generate and propagate blocks (dark circles), and buffers (triangles). In the final stage, total sum and carry bits are computed [35]. In general, KSA occupies more area and is faster than a BKA. SKA has the same delay as KSA, but it occupies a lesser area for a higher number of bits.

We implement the three architectures for 4-bit with one carry input and carry output bit. The three implementations have the same number of primary ports and bit widths. They also use three types of primitives: Bitwise XOR, AND, and OR. The maximum number of inputs per gate is two. Table 5 compares other details of the RTL implementation.

6. Results and discussion

All the experiments were executed in a Linux CPU Intel Xeon E5-2630 v4 at 2.20 GHz. This section summarizes the results of different experiments:

A. Delay estimation when training with dataset DS-I
B. Generalization of our approach by using dataset DS-II
C. Comparison of both models' performance for DS-I and DS-II
D. Evaluation of the models for larger adders
E. Comparison of our results with baseline methods

Table 6
R^2 of both models compared with STA tool results.

Dataset	Slew	Circuit	MLP R^2		TabMLP R^2	
			Delay Fall	Delay Rise	Delay Fall	Delay Rise
DS-I	✗	BKA	0.2503	0.3703	0.0827	0.5390
		KSA	0.2528	0.4596	0.1011	0.4989
		SKA	0.1113	0.6194	0.0368	0.6948
	✓	BKA	0.9295	0.9653	0.8305	0.8505
		KSA	0.9005	0.9372	0.8854	0.8549
		SKA	0.9517	0.9472	0.9057	0.8963
	✓ Prop.	BKA	0.9191	0.9635	0.8307	0.8505
		KSA	0.8888	0.9371	0.8855	0.8549
		SKA	0.9361	0.9417	0.9058	0.8890
DS-II	✓ Prop.	BKA	0.7425	0.5139	0.7785	0.7094
		KSA	0.8037	0.5902	0.7436	0.7976
		SKA	0.6996	0.6876	0.7886	0.7810

For components with a higher distance w.r.t the input ports, i.e. not directly connected to them, the input slew is considered as the predicted output slew of the connected output. To consider slew propagation, the algorithm predicts the output slew and delay sequentially, starting from the closest nodes to the input ports. Finally, our estimations are compared with results from OpenSTA using the same configuration parameters.

6.1. Results for dataset DS-I

In the following, the performance of the models MLP and TabMLP is evaluated when using DS-I as the training dataset. In particular, this experiment analyzes the effect of learning slew values simultaneously and propagating them to the next connected component in the timing path.

Table 6 compares our estimated delay values w.r.t the results from the STA tool, when using the MLP and the TabMLP architectures. Table 6 shows the R^2 value for fall and rise delay estimation. The MLP without slew information provides a low estimation accuracy. To increase the accuracy, we add slew features to the dataset. We incorporate slew estimation and propagation in our final method, as the input slew is known only for the input ports. The second last row of Table 6 confirms the strategy and shows the high R^2 values. The same experiments were conducted using TabMLP.

To evaluate the stability of the models across different input slew configurations, our models are run for both known configuration variants (V1 and V2) and one unknown but similar (V3). Table 7 shows that for V1 and V2, the results have a R^2 score higher than 80% for all adders and both models. For V3, the R^2 decreases as unknown input slews give a wrong predicted output slew, which will be propagated and affects the overall prediction of the delay. However, the MAE and MSE are still low. For simplicity, the results for rising and falling edges are averaged.

The MLP model achieves on average 6.76% MAE, 0.96% MSE, and R^2 equal to 0.87. The experiments were repeated using the TabMLP model. Table 7 shows that this model achieves on average 9.49% MAE, 1.39% MSE, and R^2 equal to 0.79.

6.2. Results for dataset DS-II

Based on the results for DS-I, we consider learning and propagating slew values as our final approach for delay prediction. We extend Table 6 with the R^2 values of MLP and TabMLP when the training set is DS-II. The results show how the TabMLP model could generalize slightly better to more RTL primitives than the MLP model.

Table 7
Evaluation of models trained with DS-I for adders with different input slews variants.

Circuit	Variant	MAE [%]		MSE [%]		R^2	
		MLP	TabMLP	MLP	TabMLP	MLP	TabMLP
BKA	V1	4.02	7.20	0.30	0.82	0.94	0.84
	V2	9.27	10.94	1.90	2.10	0.89	0.88
	V3	8.99	12.43	1.47	2.27	0.69	0.37
KSA	V1	4.92	6.98	0.46	0.70	0.91	0.87
	V2	7.17	9.32	0.94	1.19	0.92	0.90
	V3	8.01	11.40	1.015	1.87	0.79	0.59
SKA	V1	4.56	7.52	0.46	0.78	0.94	0.90
	V2	6.25	9.06	0.83	1.14	0.93	0.90
	V3	7.62	10.61	1.27	1.61	0.76	0.69
Average	*	**6.76**	9.49	**0.96**	1.39	**0.87**	0.79

To evaluate the stability of the models across different input slew configurations, we run our models for the configurations V1, V2, and V3, which were also covered in the training set. Table 8 shows that the results for different variants are similar in all circuits and models. The MLP model achieves, on average, 7.94% MAE, 0.99% MSE, and R^2 equal to 0.67. The TabMLP outperforms MLP with a 6.91% MAE, 0.68% MSE and R^2 score of 0.77 on average over the 4-bit adders.

6.3. Comparison DS-I and DS-II

In the following, the models are compared in terms of runtime and performance when training with both datasets.

6.3.1. Performance

Table 7 shows the average results for both models when trained over DS-I. The averaged R^2 coefficient for MLP is 0.87 and for TabMLP, 0.79. Thus, the averaged absolute difference for the R^2 coefficient between both models is 0.08. In this case, the best architecture is MLP. When training with DS-II, the averaged R^2 coefficients shown in Table 8 were 0.67 and 0.77 for MLP and TabMLP, respectively. Thus, TabMLP outperforms MLP by 0.1.

6.3.2. Runtime

For both datasets, the TabMLP runtime is around 9 s. TabMLP is faster as it encodes the component type in a lower D-dimensional vector. On the contrary, the MLP model uses a one-hot encoded sparse matrix. Thus, the inference time increases from 10.98 s to 16.93 s for DS-I and DS-II, respectively

In general, the quality of the predictions for the DS-II decreased compared to the results for DS-I. This is due to three factors:

(1) The more component types supported, the sparser the feature matrix is.
(2) The range of RTL features and their contribution to the prediction depends on the component type. For instance, a multiplexer has more than one input, while a NOT has only one. For the latter, the number of inputs is not a relevant feature and may disturb the learning process.
(3) The component type also impacts the slew and delay ranges. Thus, a global scaling may destroy some patterns of the objectives. For instance, logic RTL components are represented with logic cells of the technology, while arithmetic components are mapped to a set of logic gates implementing that functionality. The delay and slew values are lower for logic components, than for arithmetic operators.

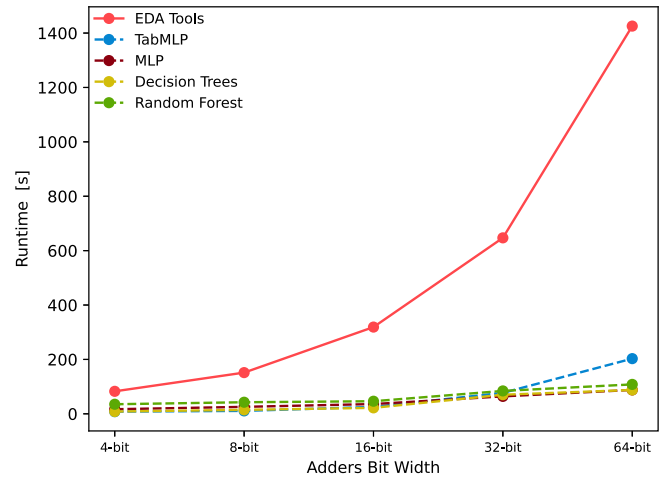


Fig. 8. Runtime comparison between ML models and EDA tools.

6.4. Results for larger adders

To evaluate the performance of the models for larger designs, we predict delays of RTL components inside 8, 16, 32, and 64-bit adders for the different STA variants.

When comparing the results for 4-bit and larger adders, the R^2 score drops from 0.77 to 0.73 for TabMLP and from 0.67 to 0.59 for MLP. For both models and larger adders, the average MSE was around 1% and a MAE around 7%. This shows that the quality of the models is slightly reduced by increasing design complexity.

Fig. 8 shows the average inference time and the time required to synthesize and run the STA tool for the same designs. On average, any ML model is 8.4× faster. For the 64-bit adders, the time benefit is more evident, as the ML models are faster by 11.5×.

6.5. Comparison with baseline methods

We compare our approach with the related works in Section 3: The modeling effort of NNs is high compared to analytical, polynomial, or macromodeling approaches due to the effort for collecting our significant dataset. Other approaches used none or fewer samples to approximate coefficients. In terms of runtime, this is lower thanks to the use of GPUs for training and inference. The gain in execution time is the main advantage of ML over other approaches. Ours and methods such as [21,25,26] need a delay characterization that uses STA tools to extract a ground truth value for the delay. This costs execution time and licenses if not open source tools are used as in our case. Analytical methods such as [18] overcome this. However, our approach is highly accurate on the test set and accurate on real designs. Finally, our approach can be extended to any RTL component, and it considers input signal transition times.

Similar to [26], this paper evaluates the performance of shallow methods for estimating pin-to-pin delays. Table 9 shows the hyperparameters for the two regressors: Decision Trees (DTs) and an RF. We train both models with DS-II. Table 8 shows the prediction results when running inference over the 4-bit adders. The deep models outperform the shallow regressors in all evaluation metrics and circuits. However, the runtimes of the four models are similar, as shown in Fig. 8.

7. Conclusion

To the best of our knowledge, our work is the first approach analyzing the delay estimation in the early stages of the design process and mapping it to a multi-objective regression task solved by NNs. We built our dataset from generated RTL and learned slew and delay

Table 8
Evaluation of models trained with DS-II for adders with different input slews variants.

Circuit	Variant	MAE [%]				MSE [%]				R^2			
		MLP	TabMLP	DT	RF	MLP	TabMLP	DT	RF	MLP	TabMLP	DT	RF
BKA	V1	6.22	5.53	7.27	7.19	0.56	0.39	1.06	0.96	0.63	0.74	0.36	39.04
	V2	8.22	8.07	10.20	9.62	0.97	0.89	1.76	1.36	0.74	0.75	0.55	63.27
	V3	9.75	8.29	9.46	9.48	1.42	0.92	1.58	1.38	0.62	0.75	0.61	63.71
KSA	V1	5.57	5.31	7.10	7.68	0.45	0.36	1.07	1.09	0.70	0.77	0.34	0.31
	V2	7.91	7.68	9.28	9.33	0.91	0.83	1.60	1.29	0.75	0.78	0.59	0.64
	V3	9.63	8.33	9.24	9.41	1.55	0.97	1.53	1.33	0.59	0.75	0.62	0.64
SKA	V1	5.89	5.19	8.16	6.47	0.51	0.35	1.38	0.75	0.69	0.78	0.51	0.54
	V2	8.64	6.79	9.74	11.45	1.11	0.67	1.75	1.84	0.72	0.83	0.55	0.52
	V3	9.57	6.99	10.12	11.21	1.41	0.69	1.89	1.78	0.63	0.82	0.52	0.53
Average	*	7.94	**6.91**	8.96	9.12	0.99	**0.68**	1.51	1.31	0.67	**0.77**	0.52	0.53

Table 9
Shallow models hyperparameters.

Regressor	Hyperparameter	Values
Random Forest	No. of Trees	400
	Min. Sample Split	2
	Min Sample Leaf	1
Decision Tree	Criterion	Squared Error
	Splitter	Best
	Min. Sample Split	2
	Min Sample Leaf	1

simultaneously to improve the performance of our models. We use two known architectures: a simple MLP and TabMLP, an MLP using an embedding layer for categorical variables. We train our models with two datasets: DS-I, which contains only logical component types, and DS-II, which supports 17 component types more.

When training with DS-I, the MLP model is the best architecture. It achieves an average R^2 of 87% taking 10.98 s. The TabMLP model is faster with 8.98 s but has a lower R^2 of 79%. The results are the opposite when generalizing to more component types. Using the dataset DS-II, the TabMLP represents the best architecture. It achieves an average R^2 of 77% while keeping the inference time around 9 s. In summary, the quality of the predictions decreases with the generalization to more component types and the increasing complexity of the evaluation circuits. However, DS-II allows us to evaluate RTL designs of real complexity. The gain of our approach using ML increases proportionally with the number of components in the design, as this slows down the synthesis and STA tools but keeps estimation times low.

In future work, we recognize four main directions. In the first place, we expect to find new ML architectures that can generalize better to different component types and circuit complexity. Second, our datasets can be extended to consider physical design information. Moreover, our solution is accurate when the transition times, technology library, and operation conditions used during training are kept constant during inference. Otherwise, new datasets have to be generated and models have to be retrained. We think that other ML techniques, such as transfer learning, can help to solve these challenges. Finally, the estimated delays can be accumulated to find critical and non-critical parts of an RTL design.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The authors do not have permission to share data.

Acknowledgment

The work described herein is partly funded by the German Federal Ministry for Economic Affairs and Energy (BMWi) as part of the research project ProgressivKI (19A21006C).

References

[1] Synopsys timing constraints and optimization user guide, 2010.
[2] G. Huang, J. Hu, Y. He, J. Liu, M. Ma, et al., Machine learning for electronic design automation: A survey, 26, Association for Computing Machinery, New York, NY, USA, 2021, no. 5.
[3] D. Sánchez Lopera, L. Servadei, V.P. Kasi, S. Prebeck, W. Ecker, RTL delay prediction using neural networks, in: 2021 IEEE Nordic Circuits and Systems Conference (NorCAS), 2021, pp. 1–7.
[4] IEEE standard for integrated circuit (IC) delay and power calculation system, IEEE Std 1481-1999, 2000.
[5] R.B. Hitchcock, G.L. Smith, D.D. Cheng, Timing Analysis of Computer Hardware, IBM J. Res. Dev. 26 (1) (1982) 100–105.
[6] Liberty user guides and reference manual suite, 2013.
[7] J.H. Zar, Spearman rank correlation, in: Encyclopedia of Biostatistics, Vol. 7, Wiley Online Library, 2005.
[8] A. Kraskov, H. Stögbauer, P. Grassberger, Estimating mutual information, Phys. Rev. E (3) 69 (6) (2004) 66138.
[9] E. Fix, J.L. Hodges, Discriminatory analysis. Nonparametric discrimination: Consistency properties, Internat. Statist. Rev. 57 (3) (1989) 238–247.
[10] G.W. Snedecor, W.G. Cochran, Statistical Methods, Vol. 1191, eight ed., Iowa State University Press, Ames, Iowa, 1991, p. 524.
[11] L.S. Shapley, A value for N-person games, RAND Corporation, Santa Monica, CA, 1952.
[12] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17), Curran Associates Inc., Red Hook, NY, USA, 2017, pp. 4768–4777.
[13] S.K. Pal, S. Mitra, Multilayer perceptron, fuzzy sets, classification, Trans. Neural Netw. 3 (5) (1992) 683–697.
[14] X. Huang, A. Khetan, M. Cvitkovic, Z. Karnin, TabTransformer: Tabular data modeling using contextual embeddings, 2020, ArXiv, [Online]. Available: https://arxiv.org/abs/2012.06678.
[15] X. Rong, Word2vec parameter learning explained, 2014, ArXiv, [Online]. Available: https://arxiv.org/abs/1411.2738.
[16] J. Devlin, M. Chang, K. Lee, K. Toutanova, BERT: Pre-training of deep bidirectional transformers for language understanding, in: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies NAACL-HLT, Vol. 1, Association for Computational Linguistics, 2019, pp. 4171–4186.
[17] J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, in: Proceedings of the 24th International Conference on Neural Information Processing Systems (NIPS'11), Curran Associates Inc., Red Hook, NY, USA, 2011, pp. 2546–2554.
[18] I. Sutherland, B. Sproull, D. Harris, Logical effort: Designing fast CMOS circuits, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1999.
[19] J. Qiu, S. Reda, S. Hassoun, Fast, accurate a priori routing delay estimation, in: Proceedings of the 12th ACM/IEEE International Workshop on System Level Interconnect Prediction (SLIP '10), Association for Computing Machinery, New York, NY, USA, 2010, pp. 77–82.

[20] D.E. Wallace, M.S. Chandrasekhar, High-level delay estimation for technology-independent logic equations, in: 1990 IEEE International Conference on Computer-Aided Design, IEEE, 1990.
[21] M. Xu, F.J. Kurdahi, Area and timing estimation for lookup table based FPGAs, in: Proceedings ED&TC European Design and Test Conference, IEEE, 1996, pp. 151–157.
[22] A. Macii, E. Macii, G. Odasso, M. Poncino, R. Scarsi, Regression-based macro-modeling for delay estimation of behavioral components, in: Proceedings of the Ninth Great Lakes Symposium on VLSI (GLS '99), IEEE Computer Society, 1999, pp. 188–191.
[23] T. Koyagi, M. Fukui, R. Saleh, Delay macromodeling and estimation for RTL, in: 2008 IEEE International Symposium on Circuits and Systems (ISCAS), 2008, pp. 2430–2433.
[24] A. Srinivasan, G.D. Huber, D.P. LaPotin, Accurate area and delay estimation from RTL descriptions, 6, (1) IEEE, 1998, pp. 168–172,
[25] M. Nemani, F.N. Najm, Delay estimation VLSI circuits from a high-level view, in: Proceedings of the 35th Annual Design Automation Conference (DAC), 1998, pp. 591–594.
[26] T. Martin, G. Grewal, S. Areibi, A machine learning approach to predict timing delays during FPGA placement, in: 2021 IEEE International Parallel and Distributed Processing Symposium Workshops, IEEE, 2021, pp. 124–127.
[27] J. Schreiner, R. Findenigy, W. Ecker, Design centric modeling of digital hardware, in: 2016 IEEE International High Level Design Validation and Test Workshop (HLDVT), 2016, pp. 46–52.
[28] F. Truyen, The Fast Guide to Model Driven Architecture, Cephas Consulting Corp, 2006.
[29] C. Wolf, Yosys open synthesis suite, 2016.
[30] Parallax static timing analyzer, 2021, [Online]. Available: https://github.com/The-OpenROAD-Project/OpenSTA.
[31] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), Association for Computing Machinery, New York, NY, USA, 2016, pp. 785–794.
[32] Pytorch-widedeep, 2021, [Online]. Available: https://github.com/jrzaurin/pytorch-widedeep.
[33] J. Bergstra, D. Yamins, D. Cox, Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures, in: International Conference on Machine Learning, 2013, pp. 115–123.
[34] A. Ghaderi, J. Frounchi, Power and speed analysis of CMOS-based multipliers using VEDIC techniques, 11, 2016, pp. 62–68,
[35] C.S. Rani, C. Ramesh, Design and implementation of high performance parallel prefix adders, 2014, pp. 5899–5903,

Daniela Sánchez Lopera received her B.Sc. degree in Electronic Engineering at the University of Antioquia, Colombia in 2016. In 2020, she received her M.Sc. degree in Electrical and Computer Engineering with focus on Embedded and Computer Systems at the Technical University of Munich. Daniela Sánchez Lopera is currently a doctoral student at the Chair of Electronic Design Automation at the Technical University of Munich and at Infineon Technologies AG in Germany. Her main research interest is the application of Machine Learning to the field of Digital Design Automation.

Lorenzo Servadei is a Senior Staff Machine Learning Engineer at Infineon Technologies AG. His main interests are methods of Reinforcement Learning applied to Design Automation of microchips. He obtained a Ph.D. degree in Computer Science from a collaboration of Infineon Technologies AG and JKU Linz. He is currently IEEE Member.

Sebastian Prebeck is an AI Hardware Accelerator Design Senior Engineer. He started his career with the Bachelor's degree (B.Sc.) in Electrical and Computer engineering at the Technical University of Munich (TUM). After graduation, he continued the education with the consecutive Master's degree (M.Sc.) at TUM. Thereby, the focus was on digital design and power engineering. Currently, Mr. Prebeck participates in a doctoral program at TUM. His research domain is on low power AI Hardware. The research is done in close cooperation with the company Infineon Technologies AG in Munich.

Wolfgang Ecker is Distinguished Engineer at Infineon and Adjunct Professor at Technical University of Munich. He is author of over 200 papers on modeling and design automation, received 5 best paper awards, and was granted with the German EDA achievement award. Wolfgang is member of Acatech, the German Academy of Science and Engineering and has been member of the AI commission of inquiry of the German Parliament. Wolfgang Ecker also coordinates the Infineon Deep Learning internal think tank.